

Ejemplos de uso de comPyLaTeX

1. Cálculo de raíces

$$f(x) = x \cdot \ln(2) \tag{1}$$

$$a_1 = -1 \tag{2}$$

$$a_2 = 10 \tag{3}$$

$$tol_{dic} = 1e - 6 \tag{4}$$

Algorithm 1 Método de dicotomía

```
while  $a_2 - a_1 > tol_{dic}$  do  
   $a_3 = (a_1 + a_2)/2$   
  if  $f(a_1) \cdot f(a_3) < 0$  then  
     $a_2 = a_3$   
  else  
     $a_1 = a_3$   
  end if  
end while  
return  $a_3$ 
```

$$a_3 = -0,000000 \tag{5}$$

2. Cálculo de soluciones de $x^a + y^b = z^c$

Valores a , b , y c

$$a = 1 \tag{6}$$

$$b = 2 \tag{7}$$

$$c = 2 \tag{8}$$

Valor máximo para las soluciones $(x, y, z \in \{1, 2, ..max\})$:

$$max = 10 \quad (9)$$

Algorithm 2 Número de soluciones

```

sols = 0
z = 1
while  $z \leq max$  do
  y = 1
  while  $y \leq max$  do
    x = 1
    while  $x \leq max$  do
      if  $x^a + y^b = z^c$  then
        sols = sols + 1
      end if
      x = x + 1
    end while
    y = y + 1
  end while
  z = z + 1
end while
return sols

```

$$sols = 5 \quad (10)$$

3. Ejemplo con ELSIFs

Algorithm 3 Ejemplo de un algoritmo con ELSIFs anidados

```

i = 10
j =  $\sqrt{i^2 + 5}$ 
if  $i \geq 5$  then
  i = i - 1
  while  $j \leq 20$  do
    j = j + log(i)
  end while
else if  $i \leq 3$  then
  i = i + 2
end if
return j

```

$$j = 21,233074 \quad (11)$$

4. Prueba con raíces cúbicas

$$a = \sqrt[3]{8} \quad (12)$$

$$a = 2,000000 \quad (13)$$

5. Cálculo: Método de Newton-Raphson

El método de Newton-Raphson permite aproximar raíces de funciones diferenciables mediante la iteración $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$. Aplicamos el método a $f(x) = x^3 - 2$, cuya raíz es $\sqrt[3]{2} \approx 1,259921$.

$$f(x) = x^3 - 2 \quad (14)$$

$$df(x) = 3 \cdot x^2 \quad (15)$$

$$x_0 = 1,5 \quad (16)$$

$$tol_{NR} = 1e - 6 \quad (17)$$

Algorithm 4 Método de Newton-Raphson para $f(x) = x^3 - 2$

$x = x_0$
while $|f(x)| > tol_{NR}$ **do**
 $x = x - f(x)/df(x)$
end while
return x

La raíz aproximada de $f(x) = x^3 - 2$ obtenida por Newton-Raphson es:

$$x = 1,259921 \quad (18)$$

6. Álgebra: Método de Jacobi

El método de Jacobi es un método iterativo para la resolución de sistemas de ecuaciones lineales. Aplicamos el método al sistema:

$$\begin{cases} 5x - y + z &= 10 \\ 2x + 8y - z &= 11 \\ -x + y + 4z &= 3 \end{cases}$$

cuya solución exacta es $(x, y, z) = (2, 1, 1)$.

Las ecuaciones de actualización de Jacobi para este sistema son:

$$x_{n+1} = \frac{10 + y_n - z_n}{5}, \quad y_{n+1} = \frac{11 - 2 \cdot x_n + z_n}{8}, \quad z_{n+1} = \frac{3 + x_n - y_n}{4}$$

$$x_j = 0 \tag{19}$$

$$y_j = 0 \tag{20}$$

$$z_j = 0 \tag{21}$$

$$tol_J = 1e - 6 \tag{22}$$

Algorithm 5 Método de Jacobi para $5x - y + z = 10$, $2x + 8y - z = 11$, $-x + y + 4z = 3$

```

 $x = (10 + y_j - z_j)/5$ 
 $y = (11 - 2 \cdot x_j + z_j)/8$ 
 $z = (3 + x_j - y_j)/4$ 
while  $|x - x_j| > tol_J$  do
     $x_j = x$ 
     $y_j = y$ 
     $z_j = z$ 
     $x = (10 + y_j - z_j)/5$ 
     $y = (11 - 2 \cdot x_j + z_j)/8$ 
     $z = (3 + x_j - y_j)/4$ 
end while
return  $x, y, z$ 

```

La solución aproximada del sistema obtenida por Jacobi es:

$$x = 2,000000 \tag{23}$$

$$y = 1,000000 \tag{24}$$

$$z = 1,000000 \tag{25}$$

7. Análisis: Método de Euler para ecuaciones diferenciales

El método de Euler explícito aproxima la solución de un problema de valor inicial $y' = f(t, y)$, $y(t_0) = y_0$, mediante la iteración:

$$y_{n+1} = y_n + h \cdot f(t_n, y_n)$$

Aplicamos el método a $y' = y$, $y(0) = 1$, cuya solución exacta es $y(t) = e^t$. Evaluamos en $t = 1$, donde la solución exacta vale $e \approx 2,718282$.

$$g(t, y) = y \quad (26)$$

$$y_0 = 1 \quad (27)$$

$$h = 0,01 \quad (28)$$

$$T = 1 \quad (29)$$

Algorithm 6 Método de Euler para $y' = y$, $y(0) = 1$

```

 $t = 0$ 
 $y = y_0$ 
while  $t < T$  do
     $y = y + h \cdot g(t, y)$ 
     $t = t + h$ 
end while
return  $y$ 

```

La aproximación de $e = y(1)$ obtenida por el método de Euler con paso $h = 0,01$ es:

$$y = 2,704814 \quad (30)$$

8. Estadística: Número combinatorio

El número combinatorio $\binom{n}{k}$ cuenta el número de subconjuntos de tamaño k de un conjunto de n elementos. Se calcula mediante:

$$\binom{n}{k} = \frac{n!}{k! \cdot (n - k)!}$$

Calculamos $\binom{10}{3} = 120$ mediante el cálculo iterativo de factoriales.

$$n = 10 \quad (31)$$

$$k = 3 \quad (32)$$

El número combinatorio $\binom{10}{3}$ calculado por el programa es:

$$\binom{10}{3} = 120,000000 \quad (33)$$

Algorithm 7 Cálculo de $\binom{n}{k}$ mediante factoriales iterativos

```
fn = 1
i = 1
while i ≤ n do
    fn = fn · i
    i = i + 1
end while
fk = 1
i = 1
while i ≤ k do
    fk = fk · i
    i = i + 1
end while
fnc = 1
i = 1
while i ≤ n - k do
    fnc = fnc · i
    i = i + 1
end while
nc = fn / (fk · fnc)
return nc
```

9. Biología matemática: Sistema de Lotka-Volterra

El sistema de Lotka-Volterra modela la dinámica de poblaciones depredador-presa:

$$\begin{cases} \frac{dx}{dt} = \alpha x - \beta xy \\ \frac{dy}{dt} = \delta xy - \gamma y \end{cases}$$

donde x es la población de presas, y la de depredadores, y $\alpha, \beta, \delta, \gamma$ son parámetros del modelo. Aplicamos Euler explícito con los parámetros clásicos $\alpha = 1,0$, $\beta = 0,1$, $\delta = 0,075$, $\gamma = 1,5$, condiciones iniciales $x(0) = 10$, $y(0) = 5$, paso $h = 0,01$ y tiempo final $T = 10$.

$$\alpha = 1,0 \tag{34}$$

$$\beta = 0,1 \tag{35}$$

$$\delta = 0,075 \tag{36}$$

$$\gamma = 1,5 \tag{37}$$

$$x_{LV} = 10,0 \tag{38}$$

$$y_{LV} = 5,0 \quad (39)$$

$$h_{LV} = 0,01 \quad (40)$$

$$T_{LV} = 10 \quad (41)$$

Algorithm 8 Euler explícito para el sistema de Lotka-Volterra

```

 $t_{LV} = 0$ 
while  $t_{LV} < T_{LV}$  do
   $dxd t = \alpha \cdot x_{LV} - \beta \cdot x_{LV} \cdot y_{LV}$ 
   $dyd t = \delta \cdot x_{LV} \cdot y_{LV} - \gamma \cdot y_{LV}$ 
   $x_{LV} = x_{LV} + h_{LV} \cdot dxd t$ 
   $y_{LV} = y_{LV} + h_{LV} \cdot dyd t$ 
   $t_{LV} = t_{LV} + h_{LV}$ 
end while
return  $x_{LV}$ 

```

Las poblaciones aproximadas en $t = 10$ obtenidas por el método de Euler son:

$$x_{LV} = 7,383307 \quad (42)$$

$$y_{LV} = 11,699475 \quad (43)$$